FINAL EXAM ANALYSIS

For CIIC 5015. By Jared Hidalgo.

JUNE 4, 2026

UNIVERSITY OF PUERTO RICO - MAYAGÜEZ

CIIC 5015: Artificial Intelligence

CONTENTS

GitHub Link: https://github.com/jaredhidalgo/CIIC-5015-Final-Exam

PROGRAMMING COMPONENT

TASK 1: OUTPUT

```
> python models.py         # Abbreviated for security.
Running shape test for AttentionBlock...
  Input  shape: torch.Size([2, 8, 16])
  Output shape: torch.Size([2, 8, 16])
Shape test passed!

Running causal mask verification...
  Causal mask verified: early positions unaffected by future token changes.

All tests passed!
```

```
> python gpt_model.py      # Abbreviated for security.
Running shape tests...

  Testing Transformer_Block...
  Transformer_Block: torch.Size([4, 16, 32]) -> torch.Size([4, 16, 32])  ✓

  Testing GPT...
  GPT forward: torch.Size([4, 16]) -> torch.Size([4, 16, 50])  ✓

  Testing generate()...
  generate: torch.Size([1, 1]) -> torch.Size([1, 11])  ✓

All shape tests passed!
```

TASK 2: CONTROL GROUP

All default values.

OUTPUT

```
Loading corpus: input.txt
  Total characters : 1,115,394
  Vocabulary size  : 65 unique characters
  Characters       : "\n !$&',-.3:;?ABCDEFGHIJKLMNOPQRSTUVWXYZa"...
  Training tokens  : 1,003,854
  Validation tokens: 111,540

Model parameters : 41,921
Training for     : 3000 steps

  Step    Train Loss    Val Loss
------------------------------------
     0        4.1743      4.1739
   300        2.5734      2.5749
   600        2.4633      2.4766
   900        2.4098      2.4364
  1200        2.3892      2.4061
  1500        2.3674      2.3890
  1800        2.3587      2.3804
  2100        2.3484      2.3687
  2400        2.3325      2.3463
  2700        2.3187      2.3447
  2999        2.3162      2.3343

============================================================
GENERATED SAMPLE (500 characters):
============================================================
Fre: mud; to so Ditoor's psind s, weriougerd s ay'st he ad lif nery trelinces Keas?
My mache hefo chirgare.


ARICETHESIO:
Girge MI she nsovesis y wayou wann:
Dit tee 'lad winciet t iss be whead peerge whe,
e co, werofpl fo, he ifopr
Toreno had we no?
Benever tsire mas merd cavath wor merunoo; teed t e, wan wo cendotoule t hano gin.

I mior ife ntean sene st then thandain mo ngan.


Whecesot Shy alare I wexpim, che bryestheld baue Re! odid st E thinghiste t
Juchil illans wicher mem;
Shy: howen s
============================================================

Training complete.
Include the loss table above and the generated sample in your analysis.pdf.
```

TASK 2: TREATMENT GROUP

Changed `MAX_ITERS` from `3000` to `30000`.

OUTPUT

```
Loading corpus: input.txt
  Total characters : 1,115,394
  Vocabulary size  : 65 unique characters
  Characters       : "\n !$&',-.3:;?ABCDEFGHIJKLMNOPQRSTUVWXYZa"...
  Training tokens  : 1,003,854
  Validation tokens: 111,540

Model parameters : 41,921
Training for     : 30000 steps

  Step    Train Loss    Val Loss
------------------------------------
     0        4.1743      4.1739
   300        2.5734      2.5749
   600        2.4633      2.4766
   900        2.4098      2.4364
  1200        2.3892      2.4061
  1500        2.3674      2.3890
  1800        2.3587      2.3804
  2100        2.3484      2.3687
  2400        2.3325      2.3463
  2700        2.3187      2.3447
  3000        2.3170      2.3349
  3300        2.2991      2.3380
  3600        2.2997      2.3302
  3900        2.2849      2.3246
  4200        2.2797      2.3136
  4500        2.2696      2.3032
  4800        2.2639      2.2870
  5100        2.2498      2.2983
  5400        2.2507      2.2890
  5700        2.2388      2.2813
  6000        2.2448      2.2856
  6300        2.2356      2.2782
  6600        2.2315      2.2774
  6900        2.2218      2.2695
  7200        2.2201      2.2723
  7500        2.2098      2.2608
  7800        2.2078      2.2714
  8100        2.2109      2.2606
  8400        2.2019      2.2496
  8700        2.1933      2.2554
  9000        2.1883      2.2491
  9300        2.1858      2.2521
  9600        2.1912      2.2442
  9900        2.1858      2.2558
 10200        2.1809      2.2466
 10500        2.1741      2.2465
 10800        2.1697      2.2330
 11100        2.1700      2.2394
 11400        2.1612      2.2313
 11700        2.1618      2.2281
 12000        2.1572      2.2346
 12300        2.1484      2.2285
```

12600	2.1515	2.2218
12900	2.1470	2.2246
13200	2.1426	2.2199
13500	2.1464	2.2199
13800	2.1507	2.2108
14100	2.1237	2.2101
14400	2.1291	2.2055
14700	2.1269	2.2107
15000	2.1251	2.2135
15300	2.1163	2.2185
15600	2.1244	2.2112
15900	2.1121	2.2063
16200	2.1056	2.1918
16500	2.1102	2.1937
16800	2.1047	2.2048
17100	2.1034	2.1917
17400	2.1091	2.1955
17700	2.1009	2.1901
18000	2.0957	2.1818
18300	2.1082	2.1865
18600	2.0838	2.1998
18900	2.0890	2.1965
19200	2.0835	2.1897
19500	2.0821	2.1802
19800	2.0770	2.1825
20100	2.0932	2.1877
20400	2.0844	2.1779
20700	2.0788	2.1817
21000	2.0614	2.1724
21300	2.0758	2.1625
21600	2.0708	2.1746
21900	2.0625	2.1575
22200	2.0775	2.1616
22500	2.0703	2.1715
22800	2.0630	2.1659
23100	2.0496	2.1734
23400	2.0540	2.1636
23700	2.0620	2.1613
24000	2.0610	2.1728
24300	2.0468	2.1674
24600	2.0563	2.1633
24900	2.0434	2.1624
25200	2.0478	2.1529
25500	2.0464	2.1520
25800	2.0418	2.1599
26100	2.0508	2.1529
26400	2.0332	2.1624
26700	2.0440	2.1544
27000	2.0333	2.1549
27300	2.0462	2.1492
27600	2.0365	2.1468
27900	2.0319	2.1514
28200	2.0270	2.1374
28500	2.0246	2.1444
28800	2.0375	2.1463
29100	2.0299	2.1536
29400	2.0215	2.1440
29700	2.0172	2.1424
29999	2.0221	2.1342

==

```
GENERATED SAMPLE (500 characters):
============================================================
Frorgth womd. Nednsw!
Clor O:
Ifult make iner as brid weath, curth me.
Gonvy bereist's; mece, anaked bangland glioom is but Qay hofaf olleld aveff durren
Mourt s APugely, grore I sthemersse, I my allllove,, toded pacesem thaindply, Aur
t cunde, at thesthat
And blougget t teittaron, balad dlay, Necrke
Thicutcoususst bak cof yoyour Nue of fownd and porro brothowin,
Ed amikse, tour me ame oin, blocow, this 'Cloysu done,
Anthis'ss
QUEEN wapp, is heffeart purpaler foreft crenow's to winduled
Ded Edin
============================================================

Training complete.
Include the loss table above and the generated sample in your analysis.pdf.
```

TASK 2 OBSERVATIONS

All verifications of Task 1 were successful.

For Task 2, I chose the Shakespeare corpus because Shakespeare's literature style is famous enough for writers to accept hypotheses on how Shakespeare would describe modern elements.

The Control Group uses the default settings while the Treatment Group increases the total training steps from 3,000 to 30,000. In the Treatment Group, the rate of change in the training and validation losses practically plateaued across these increased steps, as expected. Interestingly, the sample generated by the Treatment Group seemed to mash words together and did not focus on paragraph spacing compared to the sample generated by the Control Group.

CONCEPT REVIEW COMPONENT

ATTENTION

- Explain the five steps of your `forward()` method in order.

Step 1: Project the input into a query (Q), key (K), and value (V).

Step 2: Apply the scaled (√dk) dot product of the query (Q) and key (K) to get attention scores.

Step 3: Apply the causal mask (M) to prevent illegal information flow.

Step 4: Apply the softmax function to make a probability distribution.

Step 5: Get the weighted sum of values with V.

- Why do we divide by √dk? What happens without it?

If the key dimension (dk) is too large, then without 1/√dk, the softmax function will go into regions of extremely small gradients.

- Why must the causal mask be applied before softmax?

We want to block attention to future tokens when applying the softmax function and during autoregressive generation.

- Why use `float('-inf')` instead of 0 for masked positions?

This masks out all values in the softmax input that correspond to illegal connections.

- What shape does the scores tensor have at each step?

The scores tensor has shape `(batch_size, seq_len, layer_size)` at the end of each step.

TRANSFORMER BLOCK AND GPT

- Walk through the six steps of `Transformer_Block.forward()`.

Step 1: Apply self-attention to the input x.

Step 2: Residual connection: Add self-attention values to x.

Step 3: Apply 1st layer of normalization.

Step 4: Apply a linear feed-forward layer, then ReLU to the 1st layer of normalization as ff_out.

Step 5: Residual connection: Add ff_out to 1st layer of normalization.

Step 6: Apply 2nd layer of normalization.

- What is a residual connection and why does it matter?

Residual connections add a layer's original input to the layer's output. This addition directly contributes to learning gradients and curbs information distortion.

- Why is there no activation after the final linear layer in `GPT.forward()`?

The function already implements the softmax function to produce raw logits.

- How does `generate()` produce text autoregressively at each step?

At each step, the function crops the input by `block_size`, extracts logits, puts the logits through the softmax to get probabilities, samples the next character, and appends that sample to the sequence. The number of steps depends on the set number of new characters (`max_new_tokens`).

- Why did you choose your corpus and what did the model learn?

I chose the Shakespeare corpus because Shakespeare's literature style is famous enough for writers to accept hypotheses on how Shakespeare would describe modern elements. The model learned patterns on how to arrange characters as Shakespeare would such as words, spacing, and some punctuation. The model also generated some Shakespeare-like words, but not well enough to replicate Shakespeare.

CHALLENGE AND ANALYSIS

- Walk through one specific finding from your analysis and found limitations.

The change in maximum iterations from 3,000 to 30,000 yielded slightly lower training and validation losses, but the rate of change practically plateaued across those increased steps. One limitation from this is that increasing the number of iterations doesn't guarantee that the generated text will be better than otherwise.